Enterprise-to-Agentic (E2A)

Architecture Framework

Low-Level Abstraction Reference — Single Public Entry Point Pattern

Repository: `github.com/subhamviky/e2a-framework`
Reference implementation: `github.com/subhamviky/order-to-cash-agentic-ai`
Author: Subham Gupta, Staff Architect & AI Architect, SAP Labs India
Single Public Entry Point, Access Modifiers, API Exposure

Master Reference — Complete E2A Framework
This document is the single authoritative reference for the E2A Architecture Framework. It combines: (1) architectural principles and SAP RAP equivalence mapping, (2) universal applicability across Python/Java/Go/TypeScript, (3) NFR enforcement matrix, (4) complete class contracts with access modifiers (PUBLIC/PROTECTED/PRIVATE/CLASS VAR), (5) full method signatures with input/output parameter tables, (6) internal execution sequences with code, (7) global config key reference, and (8) implementation checklists.

1. Executive Summary

The Enterprise-to-Agentic (E2A) Architecture Framework is a formal methodology for translating enterprise distributed systems patterns — proven at $350M+ SAP TM financial settlement scale — into production-grade agentic AI systems. NFR compliance, governance, and observability are structural contracts enforced by the framework, not optional additions.

The framework defines eight abstract classes across four layers:

- Agentic Orchestration — BaseWorkflow (LangGraph graph) + BaseAgent (per-agent execution)
- Retrieval — BaseRAGPipeline (hybrid OpenSearch BM25+KNN, RAGAS evaluation gate)
- Tool Services — BaseToolService (MCP-style idempotent microservice execution)
- Foundation (environment-specific) — BaseInfraProvisioner, BaseObservability, BasePipeline, BaseGovernanceFramework

Foundation classes are defined as **Interfaces** (Python `Protocol` or pure ABC with no shared state or implementations), not abstract classes. An interface is the correct pattern when no shared implementation exists and the only purpose is to define the contract that project-specific classes must satisfy. Cloud provider, observability vendor, CI/CD platform, and governance tooling differ significantly per deployment. The interface defines the contract; the implementing class owns the complete implementation.

2. Core Architectural Principles

2.1 Template Method Pattern at Enterprise Scale

Cross-cutting concerns belong in the base class public method; domain-specific logic belongs in protected methods. Every NFR — idempotency, latency SLO, governance, token budget, observability — is enforced once. Subclasses inherit compliance without writing any additional code.

Abstract Class	Layer	PUBLIC Method	PROTECTED Methods (override to customise)
`BaseAgent`	Agentic Orchestration	`run()`	_validate_state, _build_messages, _apply_policy, _evaluate_output, _fallback, _handle_error
`BaseWorkflow`	LangGraph Orchestration	`execute()`	_build_workflow, _route_intent, _validate_workflow, _get_agent, _handle_error
`BaseRAGPipeline`	Retrieval	`retrieve()`	_chunk_text, _embed_text, _index_document, _search_index, _rerank, _generate_answer, _evaluate_answer
`BaseToolService`	Tool Microservices	`execute()`	_get_endpoint, _build_payload, _validate_input
`Foundation (×4)`	Infra / Obs / CI / Gov	`N/A — Interface, not Abstract Class`	All methods define the contract only. Implemented entirely by the project-specific class

2.2 Single Public Entry Point Rule

The Rule

PUBLIC method = orchestration contract. PROTECTED method = domain extension point. Application code NEVER calls a protected method directly. LangGraph nodes NEVER call a protected method directly. FastAPI endpoints NEVER call a protected method directly. If you are calling a protected method from outside the class, the abstraction is broken.

2.3 Access Modifier System

PUBLIC	Single public entry point. Called by application code, LangGraph nodes, and API endpoints. All orchestration logic is encapsulated here.	**PROTECTED**	Single-underscore prefix (_method). Called internally by the public method. Accessible in subclasses for override. Never called by application code directly.
PRIVATE	Double-underscore prefix (__method). Internal helpers. Never accessible in subclasses. Not shown in subclass override guide.	**CLASS VAR**	Class-level variable. Configures behaviour for the entire instance. Set before calling the public method. Default is always True for toggles.
INTERFACE	Method defined in an Interface (Protocol or pure ABC). No shared implementation exists or is expected. The implementing class owns the complete logic. Not inherited — satisfied.		

2.4 idempotency and async_mode — Class Variable Pattern

These two toggles are class-level variables with True defaults. Clear before calling the public method when the default is not desired.

```
class BaseAgent(ABC):
```

```
    idempotency: bool = True    # default ON  — clear before run() to disable
    async_mode:  bool = True    # default async — set False for sync / unit tests
```

```
# Default — both ON, nothing to set:
result = agent.run(state, config)
```

```
# Read-only agent — disable idempotency:
agent.idempotency = False
result = agent.run(state, config)
```

```
# Unit test — synchronous, no event loop:
agent.async_mode = False
result = agent.run(state, config)
```

> **Thread-Safety Note**
> Mutating class variables on a shared instance is not thread-safe under concurrent FastAPI requests. In production: instantiate a fresh agent per request (FastAPI dependency injection handles this), or store the toggle in config dict and read inside the public method via self.config.

2.5 FastAPI API Exposure Pattern

The public method maps directly to a FastAPI endpoint. No wrapper or adapter needed.

```
// Agent endpoint:
@router.post('/api/v1/agent/{agent_name}/run', response_model=AgentState)
async def run_agent(agent_name: str, request: WorkflowRequest,
                    settings: Settings = Depends(get_settings)):
    agent = AGENT_REGISTRY[agent_name](config=settings.agent_config,
                                       state=request.to_state())
    # idempotency and async_mode already True by default
    return await agent.run(state=request.to_state(), config=settings.agent_config)
```

```
// LangGraph node — same call, no wrapper:
def router_node(state: AgentState) -> AgentState:
    agent = RouterAgent(config=config, state=state)
    return agent.run(state=state, config=config)
```

2.6 SAP RAP to Agentic AI — Structural Equivalence

These are not analogies. They are the same Clean Architecture layers in two different runtimes.

Architectural Concept	SAP RAP Layer	E2A / Agentic AI Layer
Typed Global Data Model	CDS View Entity — schema enforced at DDIC compile time	AgentState TypedDict — schema enforced at type-check time
Interface Contract	Behavior Definition BDEF — declares operations; no direct instantiation	BaseAgent abstract class — declares abstract methods; ABC enforcement at class definition time
Business Logic	Behavior Implementation BIMP — implements BDEF; one per business object	Agent class with LLM + tool calls — implements BaseAgent; one per intent domain
Callable Operation	RAP Action / BOPF Action — idempotency contract, side-effect guard	MCP Tool / AgentTool — DynamoDB conditional expression for exactly-once write
Auto Quality Gate	RAP Determination — auto-triggered post-save consistency check	CriticAgent — auto-triggered groundedness gate; SLO ≥ 0.85; rewrite if below

API Exposure Layer	OData Service Binding — stateless HTTP; interface separated from behavior	FastAPI APIRouter — stateless HTTP; same separation principle

3. Universal Applicability

The E2A Framework is the GoF Template Method Pattern combined with Strategy and Chain of Responsibility, applied at enterprise scale. The same structure maps to any object-oriented runtime.

Concept	Python (E2A)	Java Spring Boot	TypeScript / Go
Base class	`ABC abstract class`	abstract class with final template method	abstract class (TS) / struct+interface (Go)
Public method	`run() / execute() / retrieve()`	final @Transactional handle()	final execute() / sealed run()
Domain hook	`@abstractmethod _method()`	abstract protected run()	abstract run() / interface method
Config injection	`config: dict in __init__`	@ConfigurationProperties bean	constructor / env config
Governance	`approved list in public method`	@Around AOP advice	middleware / decorator
Idempotency	`ensure idempotency() in base`	@Idempotent + Redis / DB check	middleware with request ID header
Async execution	`async_mode class var + aiohttp`	CompletableFuture / WebFlux Mono	Promise (TS) / goroutine (Go)
Pure contract (no impl)	`Protocol / pure ABC`	Interface	Interface (TS) / implicit interface (Go)

Spring Framework precedent:

- AbstractMessageListenerContainer — final receiveAndExecute() calls abstract executeListener(). Identical to BaseAgent.run() calling protected _build_messages() and _evaluate_output().
- AbstractJdbcTemplate — final execute() wraps connection lifecycle; subclass injects SQL action. Identical to BaseToolService.execute() wrapping HTTP lifecycle around abstract _build_payload().
- AbstractRestTemplate — final doExecute() enforces error handling; subclass injects request factory. Identical to BaseRAGPipeline.retrieve() enforcing SLOs around abstract _search_index().

4. Cross-Cutting NFR Enforcement Matrix

Every NFR is enforced exactly once inside the public method of the base class. Subclasses inherit compliance without writing any additional code.

NFR / Guardrail	BaseAgent	BaseRAG	BaseTool	Enforcement Point in Public Method
Idempotency (exactly-once)	✓	✓	✓	__ensure_idempotency() at Step 4 in run() / retrieve() / execute()
Async / Sync toggle	✓	✓	✓	async_mode class variable read inside public method

Latency SLO (< 2.5s)	✓	✓	✓	time.time() delta; raises NFRViolationError if max_latency exceeded
Retry with backoff	—	✓	✓	max_retries loop in retrieve() / execute(); tenacity for jitter
Timeout enforcement	—	✓	✓	config.timeout in aiohttp.ClientTimeout / requests session
Auth headers	—	✓	✓	auth_token injected into Authorization header before HTTP call
Governance approval	✓	✓	✓	approved_agents / approved_models / approved_tools list check at Step 2
Observability log	✓	✓	✓	Structured JSON: trace_id, request_id, latency, tokens, confidence
Token budget (FinOps)	✓	✓	✓	max_tokens config threshold; __track_tokens() at Step 6 in run()
Groundedness SLO ≥ 0.85	✓ (Critic)	✓	—	min_confidence / min_faithfulness; _fallback() triggered below threshold
Fallback / degraded mode	✓	✓	✓	_fallback() from run(); default_fallback() in resilience.py for tools
Circuit breaker hook	✓	✓	✓	pybreaker wraps public method calls in orchestration/resilience.py

BaseAgent

agents/base_agent.py — Single public method: run()

2.1 Class Variables

Access	Name	Type	Default	Purpose
CLASS VAR	`idempotency`	bool	`True`	Controls whether _ensure_idempotency() is called inside run(). Set to False before run() for read-only or exploratory agents.
CLASS VAR	`async_mode`	bool	`True`	Controls LLM invocation mode inside _llm_call(). True: asyncio coroutine. False: synchronous blocking call (testing/notebooks).

2.2 Constructor

```
def __init__(self, config: dict, state: AgentState) -> None
```

Parameter	Type	Default	Required	Description
`config`	dict	`{}`	**Yes**	Agent configuration. All keys read by run() and protected methods. See Section 7 for full key reference.
`state`	AgentState	—	**Yes**	Shared LangGraph TypedDict. Contains query, intent, retrieved_docs, response, token_usage, errors, trace_id. Mutated in-place by run() and returned.

max_latency	float	2.0	**No**	config key. Latency SLO in seconds. Enforced by run() after protected methods complete.
max_tokens	int	5000	**No**	config key. Token budget per call. Enforced via __track_tokens() inside run().
min_confidence	float	0.7	**No**	config key. Confidence threshold. If _evaluate_output() returns below this, run() calls _fallback().
approved_agents	list[str]	[]	**No**	config key. agent_name class attribute must appear here. Governance gate inside run().

2.3 PUBLIC Method — run()

run() is the ONLY public method. All orchestration is here.
Application code, LangGraph nodes, and FastAPI endpoints call run() only. Protected methods are never called from outside the class. run() accepts ALL parameters needed by ALL protected methods — no parameter is passed to a protected method that is not first passed to run().

```
# PUBLIC
def run(
    self,
    state:   AgentState,     # shared state — passed to all protected methods
    config:  dict            # config dict — passed to _apply_policy, _evaluate_output
) -> AgentState
```

Parameter	Type	Default	Required	Description
state	AgentState	—	**Yes**	Passed to every protected method that needs it. run() mutates state in-place and returns it as the final AgentState.
config	dict	—	**Yes**	Passed to _apply_policy() and _evaluate_output(). Carries domain-specific governance rules and quality thresholds.

run() — Internal Execution Sequence

run() calls ALL protected methods internally. Subclasses do not need to call any protected method. The order is fixed and enforced by the base class.

```
def run(self, state: AgentState, config: dict) -> AgentState:
    start = time.time()
    trace_id = str(uuid.uuid4())
    try:
        # Step 1 — pre-condition guard
        if not self._validate_state(state):
            raise GovernanceViolationError('Invalid agent state')
```

```
        # Step 2 — governance approval check
        if getattr(self, 'agent_name', '') not in config.get('approved_agents', []):
            raise GovernanceViolationError('Agent not approved')
```

```
        # Step 3 — domain-specific policy enforcement
        self._apply_policy(state)
```

```
        # Step 4 — idempotency (class variable toggle)
        if self.idempotency:
            self.__ensure_idempotency(state)
```

```
        # Step 5 — build LLM messages
        messages = self._build_messages(state)
```

```
        # Step 6 — token tracking (FinOps)
        token_usage = self.__track_tokens(messages)
        if token_usage > config.get('max_tokens', 5000):
            raise NFRViolationError(f'Token budget exceeded: {token_usage}')
```

```
        # Step 7 — LLM call (async or sync via class variable)
        response = self.__llm_call(messages, self.async_mode)
```

```
        # Step 8 — output quality gate
        confidence = self._evaluate_output(response, state)
        if confidence < config.get('min_confidence', 0.7):
            response = self._fallback(state)
```

```
        # Step 9 — latency SLO check
        latency = time.time() - start
        if latency > config.get('max_latency', 2.0):
            raise NFRViolationError(f'Latency SLO breached: {latency:.2f}s')
```

```
        # Step 10 — structured observability log
        logging.info({
            'agent': getattr(self, 'agent_name', self.__class__.__name__),
            'trace_id': trace_id, 'latency': latency,
            'tokens': token_usage, 'confidence': confidence
        })
```

```
        # Step 11 — state mutation and return
        state['response']    = response
        state['token_usage'] = token_usage
        state['trace_id']    = trace_id
        return state
```

```
    except Exception as e:
        return self._handle_error(e, state)
```

run() — Output

Field	Type	Description
`response`	str	Final agent response placed into state['response']
`token_usage`	int	Total tokens consumed, placed into state['token_usage']
`trace_id`	str	UUID correlation ID placed into state['trace_id']
`errors`	list	Populated by _handle_error() on exception; always present in state

2.4 PROTECTED Methods — Override to Customise

All protected methods have a default implementation of pass (empty body). Subclasses override only the methods relevant to their domain. run() calls all of them in sequence regardless — empty implementations are harmless and intentional.

Access	Signature	Purpose	Called With (from run())	Must Override?
PROTECTED	`_validate_state(state) -> bool`	Pre-condition check. Return False to abort run() with GovernanceViolationError.	`_validate_state(state)`	**Recommended**

PROTECTED	`_build_messages(state) -> list[dict]`	Build system + user message list for LLM call. Include retrieved context here for RAG agents.	`_build_messages(state)`	**Yes**
PROTECTED	`_apply_policy(state) -> None`	Agent-specific governance: tool deny list, must-cite enforcement, approval gate checks. Raise on violation.	`_apply_policy(state)`	**If governed**
PROTECTED	`_evaluate_output(response, state) -> float`	Quality score [0.0-1.0]. Below min_confidence triggers _fallback(). RouterAgent: 1.0. CriticAgent: RAGAS score.	`_evaluate_output( response, state)`	**Yes**
PROTECTED	`_fallback(state) -> dict`	Degraded-mode response. Called when confidence < min_confidence. No LLM call. Return safe static response.	`_fallback(state)`	**Recommended**
PROTECTED	`_handle_error(error, state) -> AgentState`	Exception handler. Append str(error) to state['errors']. Return state with safe defaults.	`_handle_error(e, state)`	**Recommended**

2.5 PRIVATE Methods — Internal Only

Access	Method	Purpose
PRIVATE	`_llm_call(messages, async_mode) -> dict`	Calls the configured LLM provider. Reads model_id from config. Provider-agnostic: Bedrock, Vertex AI, OpenAI. Returns {text, metadata}. Called by run() at Step 7.
PRIVATE	`__track_tokens(messages) -> int`	Counts tokens across message list. Used at Step 6 in run() for FinOps enforcement. Not overridable.
PRIVATE	`__ensure_idempotency(state) -> str`	Returns state['request_id'] if present, else generates UUID4 and writes to state. Active only when self.idempotency is True. Called at Step 4 in run().

2.6 Subclass Implementation — RouterAgent Example

Subclass only overrides the protected methods it needs. run() is NEVER redefined. All NFR enforcement, idempotency, token tracking, and logging are inherited.

```
class RouterAgent(BaseAgent):
```

```
    # Class attribute — governance key (rename-safe)
    agent_name: str = 'RouterAgent'
```

```
    # PROTECTED — override: domain-specific state check
    def _validate_state(self, state: AgentState) -> bool:
        return 'query' in state and isinstance(state['query'], str)
```

```
    # PROTECTED — override: classification prompt
    def _build_messages(self, state: AgentState) -> list[dict]:
        return [
            {'role': 'system', 'content': ROUTER_SYSTEM_PROMPT},
            {'role': 'user',   'content': state['query']}
        ]
```

```
    # PROTECTED — override: always confident in classification
    def _evaluate_output(self, response: dict,
                         state: AgentState) -> float:
        return 1.0
```

```
    # run() is NOT redefined — inherited as-is
    # _apply_policy, _fallback, _handle_error use default pass
```

```
# Caller — API endpoint or LangGraph node:
agent = RouterAgent(config=config, state=state)
# Read-only: disable idempotency for cheaper calls
agent.idempotency = False
result = agent.run(state=state, config=config)   # ONE call
```

BaseWorkflow

orchestration/base_workflow.py — Single public method: execute()

3.1 Class Variables

Access	Name	Type	Default	Purpose
CLASS VAR	`approved_workflows`	list[str]	`[]`	Governance allow-list. Workflow class name must appear here for execute() to proceed. Loaded from config in __init__.

3.2 PUBLIC Method — execute()

```
# PUBLIC
def execute(
    self,
    state:      AgentState,   # current workflow state
    state_type: type = None   # TypedDict type for compile step (optional)
) -> AgentState
```

execute() — Internal Sequence

```
def execute(self, state: AgentState, state_type: type = None) -> AgentState:
    try:
        # Step 1 — governance approval
        if self.__class__.__name__ not in self.approved_workflows:
            raise GovernanceViolationError('Workflow not approved')
```

```
        # Step 2 — build and validate LangGraph graph
        if state_type:
            workflow = self._build_workflow(state_type)
            if not self._validate_workflow(workflow):
                raise GovernanceViolationError('Invalid workflow structure')
```

```
        # Step 3 — route to correct agent
        intent = self._route_intent(state)
        agent  = self._get_agent(intent)
```

```
        # Step 4 — delegate to agent (agent.run() is the public method)
        updated_state = agent.run(state=state, config=self.config)
        return updated_state
```

```
    except Exception as e:
```

```
        return self._handle_error(e, state)
```

3.3 PROTECTED Methods

Access	Signature	Purpose	Must Override?
PROTECTED	`_build_workflow(state_type) -> CompiledGraph`	Constructs the LangGraph StateGraph: adds nodes (agent instances), edges, conditional routing functions, compiles graph.	**Yes**
PROTECTED	`_route_intent(state) -> str`	Returns canonical agent name string from state['intent']. Used by execute() to call _get_agent().	**Yes**
PROTECTED	`_validate_workflow(workflow) -> bool`	Returns True if graph structure is valid. Called after _build_workflow() inside execute().	**Recommended**
PROTECTED	`_get_agent(intent: str) -> BaseAgent`	Returns instantiated agent for the given intent string. Registry lookup. @abstractmethod — enforced at class definition time.	**Yes**
PROTECTED	`_handle_error(error, state) -> AgentState`	Exception handler. Returns safe state with error appended to state['errors'].	**Recommended**

BaseRAGPipeline

rag/base_rag_pipeline.py — Single public method: retrieve()

4.1 Class Variables

Access	Name	Type	Default	Purpose
CLASS VAR	`idempotency`	bool	`True`	When True, _ensure_idempotency() generates a stable request_id from query hash inside retrieve(). Set False for non-idempotent exploratory retrieval.
CLASS VAR	`async_mode`	bool	`True`	When True, vector store search is dispatched asynchronously. When False, synchronous (testing, offline indexing).

4.2 PUBLIC Method — retrieve()

```
# PUBLIC
async def retrieve(
    self,
    query:    str,            # raw query string
    top_k:    int = 5,        # candidate document count
    text:     str = None,     # document text for indexing (optional)
    doc_id:   str = None,     # document ID for indexing (optional)
    metadata:dict = None      # document metadata for indexing (optional)
) -> list[dict]
```

Parameter	Type	Default	Required	Description

query	str	—	**Yes**	Raw query. Passed to _embed_text() to produce the query vector.
top_k	int	5	**No**	Number of documents to retrieve from _search_index(). Passed to _rerank() as well.
text	str | None	None	**No**	If provided, retrieve() calls _chunk_text() + _embed_text() + _index_document() before search. Enables on-the-fly indexing.
doc_id	str | None	None	**No**	Document ID for indexing. Required when text is provided.
metadata	dict | None	None	**No**	Metadata for the document being indexed. Passed to _index_document().

retrieve() — Internal Sequence

```
async def retrieve(self, query, top_k=5,
                   text=None, doc_id=None, metadata=None) -> list[dict]:
    start = time.time()
    try:
        # Step 1 — optional on-the-fly document indexing
        if text and doc_id:
            chunks     = self._chunk_text(text)
            embeddings = [self._embed_text(c) for c in chunks]
            self._index_document(doc_id, text, metadata or {})
```

```
        # Step 2 — idempotency (class variable)
        if self.idempotency:
            request_id = self.__ensure_idempotency(query)
```

```
        # Step 3 — auth header
        headers = {'Authorization': f'Bearer {self.auth_token}'} \
                  if self.auth_token else {}
```

```
        # Step 4 — embed query and search (async or sync)
        query_vector = self._embed_text(query)
        for attempt in range(self.max_retries):
            try:
                docs = await self.__async_search(query_vector, top_k, headers) \
                       if self.async_mode \
                       else self._search_index(query_vector, top_k)
                break
            except Exception as e:
                if attempt == self.max_retries - 1: raise
```

```
        # Step 5 — rerank
        docs = self._rerank(query, docs, top_k)
```

```
        # Step 6 — generate answer (optional — empty by default)
        answer = self._generate_answer(query, [d['content'] for d in docs])
```

```
        # Step 7 — evaluate answer (optional — empty by default)
        if answer:
            eval_result = self._evaluate_answer(query, answer,
                                [d['content'] for d in docs])
            if eval_result.get('faithfulness', 1.0) < self.min_faithfulness:
                raise NFRViolationError('Faithfulness SLO breached')
```

```
        # Step 8 — latency SLO
        if time.time() - start > self.max_latency:
            raise NFRViolationError('Latency SLO breached')
```

```
        # Step 9 — observability log
```

```
        logging.info({'pipeline': self.__class__.__name__,
                      'docs': len(docs), 'latency': time.time()-start})
        return docs
    except Exception as e:
        logging.error(f'RAG retrieve failed: {e}'); return []
```

4.3 PROTECTED Methods

Access	Signature	Purpose	Must Override?
PROTECTED	`_chunk_text(text) -> list[str]`	Split text into chunks. Strategy: fixed-window, overlap, sentence-boundary, or semantic. Called when text is passed to retrieve().	**Yes**
PROTECTED	`_embed_text(text) -> list[float]`	Generate dense embedding vector. Provider-agnostic: Bedrock Titan, Vertex textembedding-gecko, sentence-transformers, OpenAI.	**Yes**
PROTECTED	`_index_document(doc_id, text, metadata) -> None`	Persist chunks + embeddings to vector store. Called when text+doc_id are passed to retrieve().	**If indexing needed**
PROTECTED	`_search_index(query_vector, top_k) -> list[dict]`	Vector store search call. KNN, BM25, or hybrid. Called at Step 4 inside retrieve().	**Yes**
PROTECTED	`_rerank(query, docs, top_k) -> list[dict]`	Reorder candidate docs. Default: sort by score. Override for cross-encoder or Cohere Rerank. Governance model check inside retrieve().	**If reranking needed**
PROTECTED	`_generate_answer(query, contexts) -> str`	LLM generation from retrieved contexts. Empty by default (pass). Override in agents that generate a response inside the RAG cycle.	**If answer needed**
PROTECTED	`_evaluate_answer(query, answer, contexts) -> dict`	RAGAS or equivalent evaluation. Returns {faithfulness, relevancy}. Empty by default. Override when min_faithfulness SLO must be enforced.	**If eval needed**

BaseToolService

tools/base_tool.py — Single public method: execute()

5.1 Class Variables

Access	Name	Type	Default	Purpose
CLASS VAR	`idempotency`	bool	`True`	When True, execute() calls __ensure_idempotency(payload). Critical for WRITE tools. Set False for READ tools (check_stock, risk_score).
CLASS VAR	`async_mode`	bool	`True`	When True, execute() uses aiohttp for the HTTP call. When False, uses requests (synchronous). Set False for unit testing.

5.2 PUBLIC Method — execute()

```
# PUBLIC
async def execute(
    self,
    payload:  dict   # raw input data for this tool
) -> dict
```

Parameter	Type	Default	Required	Description
payload	dict	—	**Yes**	Raw input. Passed to _validate_input(), then transformed by _build_payload() before the HTTP call. May receive request_id injected by __ensure_idempotency().

execute() — Internal Sequence

```
async def execute(self, payload: dict) -> dict:
    start = time.time()
    try:
        # Step 1 — governance approval
        if not self.__is_tool_approved():
            raise GovernanceViolationError('Tool not approved')
```

```
        # Step 2 — domain-specific input validation
        if not self._validate_input(payload):
            raise ValidationError('Invalid tool input')
```

```
        # Step 3 — idempotency (class variable toggle)
        if self.idempotency:
            self.__ensure_idempotency(payload)
```

```
        # Step 4 — auth header
        headers = {'Authorization': f'Bearer {self.auth_token}'} \
                  if self.auth_token else {}
```

```
        # Step 5 — build request body
        body     = self._build_payload(payload)
        endpoint = self._get_endpoint()
```

```
        # Step 6 — HTTP call (async or sync)
        for attempt in range(self.max_retries):
            try:
                if self.async_mode:
                    async with aiohttp.ClientSession(
                        timeout=aiohttp.ClientTimeout(total=self.timeout)
                    ) as session:
                        async with session.post(
                            endpoint, json=body, headers=headers
                        ) as resp:
                            resp.raise_for_status()
                            result = await resp.json()
                else:
                    r = requests.post(endpoint, json=body,
                                      headers=headers, timeout=self.timeout)
                    r.raise_for_status(); result = r.json()
                break
            except Exception as e:
                if attempt == self.max_retries - 1: raise
```

```
        # Step 7 — latency SLO
        latency = time.time() - start
        if latency > self.max_latency:
```

```
            raise NFRViolationError(f'Tool latency SLO breached: {latency:.2f}s')
```

```
        # Step 8 — observability log
        logging.info({'tool': getattr(self, 'tool_name', '?'),
                      'latency': latency, 'endpoint': endpoint})
        return result
    except Exception as e:
        return {'success': False, 'error': str(e)}
```

5.3 PROTECTED Methods

Access	Signature	Purpose	Must Override?
PROTECTED	`_get_endpoint() -> str`	Returns HTTP URL. Read from config['tool_base_url'] for environment portability. Same subclass works local, Lambda, Cloud Run.	**Yes**
PROTECTED	`_build_payload(payload) -> dict`	Transform input dict to tool API request schema. Field mapping and type coercion. Tool-specific.	**Yes**
PROTECTED	`_validate_input(payload) -> bool`	Domain-specific validation: required fields, types, business rules. Return False to abort execute() with ValidationError.	**Yes**

5.4 PRIVATE Methods

Access	Method	Purpose
PRIVATE	`__ensure_idempotency(payload) -> str`	Reuses payload['request_id'] if present; generates UUID4 and writes it if absent. Stable identity for exactly-once write guarantee.
PRIVATE	`__is_tool_approved() -> bool`	Reads self.tool_name class attribute against config['approved_tools']. Returns False if not present. Rename-safe — uses attribute not class name.

5.5 Subclass Example — check_stock (READ tool)

```
class CheckStockTool(BaseToolService):
```

```
    # Class attribute — governance key
    tool_name: str = 'check_stock'
```

```
    # PROTECTED — override
    def _get_endpoint(self) -> str:
        return self.config['tool_base_url'] + '/check_stock'
```

```
    def _build_payload(self, payload: dict) -> dict:
        return {'product_id': payload['product_id'],
                'warehouse_id': payload.get('warehouse_id', 'WH-001')}
```

```
    def _validate_input(self, payload: dict) -> bool:
        return 'product_id' in payload
```

```
# Caller — READ tool: idempotency not needed
tool = CheckStockTool(config=config)
tool.idempotency = False
result = await tool.execute(payload={'product_id': 'SKU-001'})
```

Foundation Layers

Infrastructure · Observability · CI/CD · Governance — Interface Pattern (not Abstract Class)

All Methods are Interface Contracts — No Shared Implementation

Foundation classes are defined as **Interfaces**, not Abstract Classes. An abstract class is correct when the base class provides shared concrete implementations that subclasses inherit (BaseAgent, BaseRAGPipeline, BaseToolService — which all have shared public method bodies enforcing NFRs). An interface is correct when the base defines a pure contract with no shared implementation — which is exactly the case for infrastructure, observability, CI/CD, and governance. These differ so significantly per cloud provider and project that no default implementation would be correct. Python implements interfaces via Protocol (structural/duck typing) or via ABC with all @abstractmethod and no __init__ state. The distinction matters: using an abstract class where an interface is correct misleads the reader into expecting shared behaviour that does not exist.

6.1 BaseInfraProvisioner

class BaseInfraProvisioner(ABC):

Contract	Method Signature	Purpose	Subclass Example
INTERFACE	`_define_network() -> None`	Network topology definition	Terraform VPC + subnets / GCP VPC / Azure VNet
INTERFACE	`_define_compute() -> None`	Compute resource definition	ECS Fargate / GKE / Cloud Run / AKS
INTERFACE	`_define_storage() -> None`	Storage and database definition	DynamoDB / Firestore / Spanner / CosmosDB
INTERFACE	`_define_secrets() -> None`	Secret management definition	AWS Secrets Manager / GCP Secret Manager / Azure Key Vault
INTERFACE	`_apply_infra() -> None`	Execute provisioning	terraform apply / CDK deploy / Pulumi up

```
# Python — using Protocol for structural interface (duck typing, no inheritance required)
class BaseInfraProvisioner(Protocol):
# Alternative: ABC with all @abstractmethod and no __init__ (nominal interface)
class BaseInfraProvisioner(ABC): # No __init__, no shared state, no concrete methods — pure contract
```

6.2 BaseObservability

class BaseObservability(ABC):

Contract	Method Signature	Purpose	Subclass Example
INTERFACE	`_collect_metrics() -> None`	Custom metric emission	CloudWatch PutMetricData / Datadog / GCP Monitoring
INTERFACE	`_collect_traces() -> None`	Distributed trace emission	AWS X-Ray / Jaeger / GCP Cloud Trace / Zipkin
INTERFACE	`_collect_logs() -> None`	Structured log shipping	CloudWatch Logs / Datadog / GCP Logging / ELK
INTERFACE	`_enforce_slo(latency, avail) -> None`	SLO violation detection and alerting	CloudWatch Alarms / PagerDuty / GCP Alert Policies

```
# Python — using Protocol for structural interface (duck typing, no inheritance required)
class BaseObservability(Protocol):
# Alternative: ABC with all @abstractmethod and no __init__ (nominal interface)
class BaseObservability(ABC): # No __init__, no shared state, no concrete methods — pure contract
```

6.3 BasePipeline

class BasePipeline(ABC):

Contract	Method Signature	Purpose	Subclass Example
INTERFACE	`_run_tests() -> None`	Unit and integration test gate	pytest / JUnit / Go test + coverage threshold
INTERFACE	`_run_rag_eval() -> None`	RAG quality gate	RAGAS faithfulness ≥ 0.85 enforcement
INTERFACE	`_build_artifacts() -> None`	Build, package, and scan	Docker build + ECR push + Trivy scan
INTERFACE	`_deploy_service() -> None`	Service deployment	ECS rolling deploy / Cloud Run / Kubernetes apply
INTERFACE	`_enforce_policy() -> None`	Policy compliance gate	YAML validation / OPA check / Bandit SAST

```
# Python — using Protocol for structural interface (duck typing, no inheritance required)
class BasePipeline(Protocol):
# Alternative: ABC with all @abstractmethod and no __init__ (nominal interface)
class BasePipeline(ABC): # No __init__, no shared state, no concrete methods — pure contract
```

6.4 BaseGovernanceFramework

class BaseGovernanceFramework(ABC):

Contract	Method Signature	Purpose	Subclass Example
INTERFACE	`_apply_policy(payload) -> None`	Policy enforcement and PII redaction	governance.yaml OPA + regex redaction pipeline
INTERFACE	`_track_costs(cost) -> None`	FinOps cost recording per workflow	CloudWatch WorkflowCostUSD / BigQuery cost table
INTERFACE	`_enforce_slo(metrics) -> None`	SLO evaluation and error budget tracking	Error budget calc; deployment freeze on breach
INTERFACE	`_circuit_breaker(failures) -> None`	Circuit breaker state management	pybreaker state transitions + CloudWatch metric

```
# Python — using Protocol for structural interface (duck typing, no inheritance required)
class BaseGovernanceFramework (Protocol):
# Alternative: ABC with all @abstractmethod and no __init__ (nominal interface)
class BaseGovernanceFramework (ABC): # No __init__, no shared state, no concrete methods — pure contract
```

7. Global Config Keys Reference

All config keys are injected via the config: dict constructor argument. No subclass or base class code changes required to modify any value. The same subclass runs across dev, staging, and production with only config dict changes.

Key	Used By	Type	Default	Controls
▼ Behaviour Toggles (class variables — set before calling public method)				
`idempotency`	Agent, RAG, Tool	bool	`True`	Enable/disable request deduplication. Default True. Clear before calling public method for read-only calls.
`async_mode`	Agent, RAG, Tool	bool	`True`	True: async (aiohttp / asyncio). False: synchronous. Same subclass, same config, different mode.
▼ SLO Thresholds				
`max_latency`	Agent, RAG, Tool	float	`2.0 / 2.5`	Latency SLO in seconds. 2.0 for agents/tools; 2.5 for RAG. Enforced inside public method.
`max_tokens`	Agent, RAG	int	`5000`	Token budget per workflow. Enforced inside run() via __track_tokens().
`min_confidence`	Agent	float	`0.7`	Confidence gate. Below this: _fallback() called inside run().
`min_faithfulness`	RAG	float	`0.85`	Groundedness SLO. Enforced inside retrieve() after _evaluate_answer().
▼ Resilience				
`max_retries`	RAG, Tool	int	`3`	Retry count on transient failures inside retrieve() / execute().
`timeout`	RAG, Tool	float	`5.0 / 10.0`	HTTP session timeout in seconds.
▼ Provider & Model Selection				
`model_id`	Agent	str	`'...'`	LLM model. Provider-agnostic: Bedrock ARN / Vertex model name / OpenAI model name. Read by __llm_call().
`embed_model`	RAG	str	`'...'`	Embedding model. Swap providers via this key only. Read by _embed_text().
`reranker_model`	RAG	str	`'...'`	Reranker model. Must be in approved_models. Read by _rerank().
`tool_base_url`	Tool	str	`'http://...'`	Base URL for tool microservice. Swap local/Lambda/Cloud Run by changing this. Read by _get_endpoint().
▼ Governance				
`approved_agents`	Agent	list	`[]`	agent_name class attribute must appear here. Block any agent by removing its name.
`approved_models`	RAG	list	`[]`	embed_model and reranker_model must be present. Block models by removal.

`approved_tools`	Tool	list	`[]`	tool_name class attribute must appear here. Block any tool by removing its name.
`auth_token`	RAG, Tool	str|None	`None`	Bearer token for Authorization header. None = no auth (local dev).

8. Implementation Checklist

New Agent Subclass

- Define class attribute: agent_name = 'MyAgentName' — stable governance key
- Add agent_name to config['approved_agents'] in the workflow config
- Override _validate_state(state) — return bool for required field check
- Override _build_messages(state) — return [{role, content}] list
- Override _evaluate_output(response, state) — return float [0.0-1.0]
- Override _apply_policy(state) if the agent has governance rules — raise on violation
- Override _fallback(state) — return safe dict without LLM call
- Override _handle_error(error, state) — append to state['errors'], return safe state
- DO NOT override run() unless the orchestration sequence itself must change
- DO NOT call any protected method from outside the class
- Set agent.idempotency = False before run() for read-only agents (e.g., CriticAgent)
- Set agent.async_mode = False for unit tests (synchronous, no event loop required)

New RAG Pipeline Subclass

- Call super().__init__(config) with complete config dict including all required keys
- Add embed_model and reranker_model to config['approved_models']
- Override _embed_text(text) — call chosen embedding provider SDK
- Override _search_index(query_vector, top_k) — vector store SDK search call
- Override _chunk_text(text) — define chunking strategy
- Override _index_document(doc_id, text, metadata) if indexing is needed
- Override _rerank(query, docs, top_k) if cross-encoder or Cohere reranking is needed
- Override _generate_answer(query, contexts) and _evaluate_answer() if RAG answer generation is needed
- Call pipeline.retrieve() from agent code only — never _search_index() or _embed_text() directly

New Tool Subclass

- Define class attribute: tool_name = 'my_tool' — stable governance key
- Add tool_name to config['approved_tools']
- Override _get_endpoint() — read from config['tool_base_url'] for environment portability
- Override _build_payload(payload) — transform to tool API schema
- Override _validate_input(payload) — required fields, types, business rules
- For WRITE tools: leave tool.idempotency = True (default)
- For READ tools: set tool.idempotency = False before execute()
- Set tool.async_mode = False for unit tests (no aiohttp event loop)
- Call tool.execute(payload) from agent code only — never _get_endpoint() or _build_payload() directly